

FDS Project 1(Hard): Performance Measurement (A+B)

Anonymous Author

Date: 2026-03-12

Given S as a collection of N positive integers that are no more than V . For any given number c , you are supposed to find two integers a and b from S , so that $a + b = c$.

Your tasks are:

- (1) Implement at least two different algorithms to solve this problem;
- (2) Analyze the complexities of the two algorithms;
- (3) Measure and compare the performances of the two algorithms for V and $N = 1000, 5000, 10000, 20000, 40000, 60000, 80000, 100000$.

1. Introduction

I wrote the program containing two algorithms, but found that the first algorithm with a time complexity of $O(n^2)$ actually runs with a **linearly increasing** runtime as N grows, rather than quadratically.

I realized that, take the number 100 as an example—it can be written as 50+50, 49+51, and so on down to 1+99, giving 50 combinations. As the number N increases, the number of combinations also grows.

Therefore, Algorithm 1 can easily find the answer within its **first few iterations**, thus failing to exhibit the $O(n^2)$ complexity. As shown in the picture below:

```

9
10 /*
11 Given S as a collection of N positive integers
12 that are no more than V. For any given number c,
13 you are supposed to find two integers a and b
14 from S, so that a+b=c.
15 */
16
17 int find1(int *s, int N, int V, int c, int *a, int *b){
18     //the first algorithm, time complexity O(n^2).
19     for(int i = 0; i < N; i++){
20         for(int j = i+1; j < N; j++){
21             //brute-force enumeration.
22             if(s[i]+s[j] == c){
23                 *a = s[i], *b = s[j];
24                 return 1;
25             }
26         }
27     }
28     return 0;
29     //fail
30 }
31
32

```

```

Algorithm1, N = 1000, Tick: 44, Time: 0.044000
Algorithm2, N = 1000, Tick: 25, Time: 0.025000

Algorithm1, N = 5000, Tick: 240, Time: 0.240000
Algorithm2, N = 5000, Tick: 115, Time: 0.115000

Algorithm1, N = 10000, Tick: 379, Time: 0.379000
Algorithm2, N = 10000, Tick: 230, Time: 0.230000

Algorithm1, N = 20000, Tick: 888, Time: 0.888000
Algorithm2, N = 20000, Tick: 454, Time: 0.454000

Algorithm1, N = 40000, Tick: 1474, Time: 1.474000
Algorithm2, N = 40000, Tick: 918, Time: 0.918000

Algorithm1, N = 60000, Tick: 2739, Time: 2.739000
Algorithm2, N = 60000, Tick: 1350, Time: 1.350000

Algorithm1, N = 80000, Tick: 3072, Time: 3.072000
Algorithm2, N = 80000, Tick: 1797, Time: 1.797000

Algorithm1, N = 100000, Tick: 5878, Time: 5.878000
Algorithm2, N = 100000, Tick: 2213, Time: 2.213000

请按任意键继续. . .

```

Image 1

So that we should use **boundary cases** to demonstrate the time complexity!

Just make $c = 1$ or 2 in Algorithm 1:

```

D:\FDS\project1.exe
Algorithm1, N = 1000, Tick: 64, Time: 0.064000
Algorithm2, N = 1000, Tick: 6, Time: 0.006000

Algorithm1, N = 5000, Tick: 1449, Time: 1.449000
Algorithm2, N = 5000, Tick: 10, Time: 0.010000

Algorithm1, N = 10000, Tick: 4399, Time: 4.399000
Algorithm2, N = 10000, Tick: 20, Time: 0.020000

Algorithm1, N = 20000, Tick: 21067, Time: 21.067000
Algorithm2, N = 20000, Tick: 47, Time: 0.047000

Algorithm1, N = 40000, Tick: 75586, Time: 75.586000
Algorithm2, N = 40000, Tick: 90, Time: 0.090000

Algorithm1, N = 60000, Tick: 112814, Time: 112.814000
Algorithm2, N = 60000, Tick: 136, Time: 0.136000

Algorithm1, N = 80000, Tick: 169293, Time: 169.293000
Algorithm2, N = 80000, Tick: 173, Time: 0.173000

Algorithm1, N = 100000, Tick: 152545, Time: 152.545000
Algorithm2, N = 100000, Tick: 222, Time: 0.222000

请按任意键继续 . . .

```

Image 2

In the picture **above**, we can see that the program is able to **reflect the quadratic and linear complexity** of the two algorithms respectively. However, the tick values are **too small (6)** and **too large (169293)**.

This leads to large errors (> 10%) when the tick is too small, while a very large tick wastes a lot of time.

Therefore, we need to dynamically adjust the value of K to keep the tick of each test case within a reasonable range:

```

1  /*Algorithm 1*/
2  K = 120000/N+1; //setting a proper K to keep total time in a reasonable range
3                  //"+1" is to prevent k=0, which would result in NaN!
4  c = 1;          //providing boundary cases to demonstrate the time complexity!
5  start = clock(); //repeating the algorithm for K times
6
7  /*Algorithm 2*/
8  K = 1000000/N+1; //this one is faster, so repeating more times to improve accuracy

```

As shown in the picture below:

```

D:\FDS\project1.exe
Algorithm1, N = 1000, K = 121, Tick: 120, Total_time: 0.120, Duration: 0.000992
Algorithm2, N = 1000, K = 10001, Tick: 240, Total_time: 0.240, Duration: 0.000024

Algorithm1, N = 5000, K = 25, Tick: 588, Total_time: 0.588, Duration: 0.023520
Algorithm2, N = 5000, K = 2001, Tick: 219, Total_time: 0.219, Duration: 0.000109

Algorithm1, N = 10000, K = 13, Tick: 1224, Total_time: 1.224, Duration: 0.094154
Algorithm2, N = 10000, K = 1001, Tick: 226, Total_time: 0.226, Duration: 0.000226

Algorithm1, N = 20000, K = 7, Tick: 2654, Total_time: 2.654, Duration: 0.379143
Algorithm2, N = 20000, K = 501, Tick: 233, Total_time: 0.233, Duration: 0.000465

Algorithm1, N = 40000, K = 4, Tick: 6075, Total_time: 6.075, Duration: 1.518750
Algorithm2, N = 40000, K = 251, Tick: 219, Total_time: 0.219, Duration: 0.000873

Algorithm1, N = 60000, K = 3, Tick: 10243, Total_time: 10.243, Duration: 3.414333
Algorithm2, N = 60000, K = 167, Tick: 230, Total_time: 0.230, Duration: 0.001377

Algorithm1, N = 80000, K = 2, Tick: 12108, Total_time: 12.108, Duration: 6.054000
Algorithm2, N = 80000, K = 126, Tick: 225, Total_time: 0.225, Duration: 0.001786

Algorithm1, N = 100000, K = 2, Tick: 18907, Total_time: 18.907, Duration: 9.453500
Algorithm2, N = 100000, K = 101, Tick: 228, Total_time: 0.228, Duration: 0.002257

请按任意键继续 . . .

```

Image 3

2. Algorithm Specification

2.1 Implement the two functions (30 pts.)

```
1  int find1(int *s, int N, int V, int c, int *a, int *b){//the first algorithm, time
    complexity O(n^2).
2      for(int i = 0; i < N; i++){
3          for(int j = i+1; j < N; j++){//brute-force enumeration.
4              if(s[i]+s[j] == c){
5                  *a = s[i], *b = s[j];
6                  return 1;//success
7              }
8          }
9      }
10     return 0;//fail
11 }
```

```
1  int find2(int *s, int N, int V, int c, int *a, int *b){//the second algorithm, time
    complexity O(n).
2      int *flag = (int *)calloc((V+5), sizeof(int));
3      //using a flag array to mark number occurrences
4      //using calloc so that all elements are initialized to 0
5      for(int i = 0; i < N; i++) flag[s[i]]++;
6      for(int i = c/2; i>=0&& c-i<=V; i--){
7          //start from the middle part of the flag array,
8          //making it easier to avoid out-of-bounds.
9              if(flag[i]&&flag[c-i]){
10                 if(i == c-i&&flag[i]<2) continue;
11                 *a = i, *b = c-i;
12                 free(flag);
13                 return 1;
14             }
15         }
16     free(flag);
17     return 0;
18 }
```

2.2 testing program (20 pts.) with sufficient comments.

- For the detailed content of the program, see the **Appendix** section.^[^1]

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<time.h>
4  ...
```

3. Testing Results

3.1 Decide the iteration number K for each test case and fill in the table of results (8 pts.)

- For the content related to the generation of K , see the **Introduction** section.
- We only list the **result table** here.

V = 100000	N	1000	5000	10000	20000	40000	60000	80000	100000
Algorithm1	iterations(K)	121	25	13	7	4	3	2	2
	Ticks	120	588	1224	2654	6075	10243	12108	18907
	Total Time(sec)	0.120	0.588	1.224	2.654	6.075	10.243	12.108	18.907
	Duration(sec)	0.000992	0.023520	0.094154	0.379143	1.518750	3.414333	6.054000	9.453500
Algorithm2	iterations(K)	10001	2001	1001	501	251	167	126	101
	Ticks	240	219	226	233	219	230	225	228
	Total Time(sec)	0.240	0.219	0.226	0.233	0.219	0.230	0.225	0.228
	Duration(sec)	0.000024	0.000109	0.000226	0.000465	0.000873	0.001377	0.001786	0.002257

3.2 Plot the run times of the functions (12 pts.)

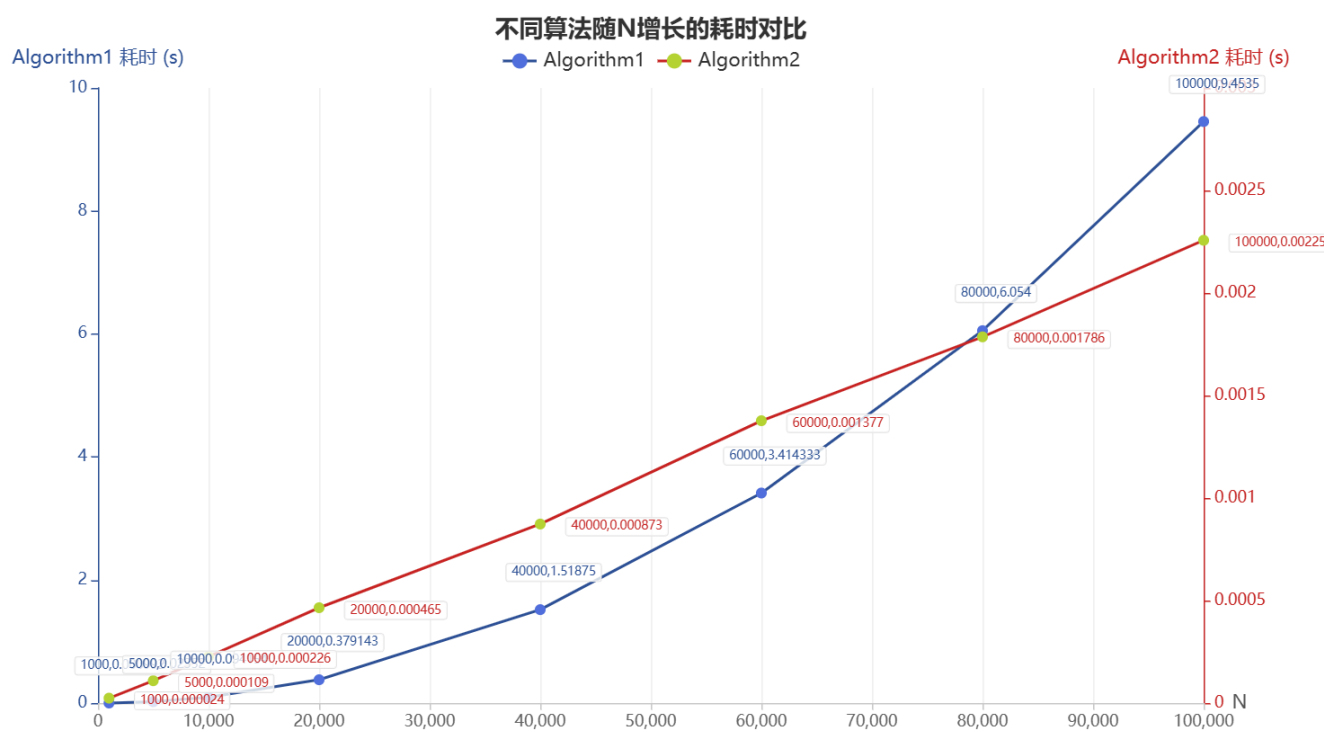


Image 4

- Based on the line graph of run times, Algorithm 1 grows quadratically overall, and Algorithm 2 grows linearly.

4. Analysis and Comment(10 pts.)

4.1 Analysis of the time and space complexities of the algorithms

4.1.1 Algorithm 1 (Brute-force Enumeration)

- The algorithm uses nested loops to traverse all pairs of elements in the array: the outer loop runs N times, and the inner loop runs $N - 1, N - 2, \dots, 1$ times for each outer loop iteration. In the worst case (no valid $a + b = c$ pair), the total number of iterations is $\frac{N(N-1)}{2}$. Time complexity: $O(N^2)$.
- The space complexity is $O(1)$ as it only uses a fixed number of temporary variables (no additional dynamic memory).

4.1.2 Algorithm 2 (Hash-based Marking)

- The algorithm consists of two linear passes:
 - (1) a single loop to mark the occurrence of each element in the array ($O(N)$)
 - (2) a single loop to search for valid pairs in the flag array ($O(V)$).
- Overall time complexity: $O(N)$.

Important

Overall time complexity will be $O(N + V)$ if V is variable. ¹

- The space complexity is $O(V)$ due to the dynamically allocated flag array of size $V + 5$.

4.2 Comments and further possible improvements

4.2.1 Test Design Rationality

4.2.1.1 Boundary Case Selection

Using $c = 1$ (unsolvable) can reveal the true time complexity of both algorithms.

4.2.1.2 Dynamic Iteration Count (K)

Adjusting K as $K = C/N + 1$ solves the precision issue of **duration** (which has limited resolution): repeating fast algorithms (Algorithm 2) more times amplifies the runtime, reducing relative error.

4.2.1.3 Large V (100000)

A small V would limit the range of integers in S , leading to frequent early termination of Algorithm 1; using $V = 100000$ ensures the array S has sufficient diversity.

4.2.2 Limitations and Improvements

4.2.2.1 Number generation function `Random()`

The process of generating random numbers itself occupies a period of time, which slightly affects the result accuracy.

4.2.2.2 Varying V

The experiment fixed $V = 100000$.

Testing different V values would reveal how Algorithm 2's space complexity ($O(V)$) and time complexity $O(N + V)$ impacts performance.

5. Appendix: Source Code (in C)

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<time.h>
4
5  clock_t start, stop;
6  int s[100005];
7  int N, V, c;
8
9  /*
10 Given s as a collection of N positive integers
11 that are no more than V. For any given number c,
12 you are supposed to find two integers a and b
13 from s, so that a+b=c.
14 */
15
16 int find1(int *s, int N, int V, int c, int *a, int *b){
17     //the first algorithm, time complexity O(n^2).
18     for(int i = 0; i < N; i++){
19         for(int j = i+1; j < N; j++){
20             //brute-force enumeration.
21             if(s[i]+s[j] == c){
22                 *a = s[i], *b = s[j];
23                 //printf("%d + %d = %d\n",s[i], s[j], c);
24                 return 1;
25                 //success
26             }
27         }
28     }
29     //printf("F\n");
30     return 0;
31     //fail
32 }
33
34 int find2(int *s, int N, int V, int c, int *a, int *b){
35     //the second algorithm, time complexity O(n).
```

```

36     int *flag = (int *)calloc((v+5), sizeof(int));
37     //using a flag array to mark number occurrences
38     //using calloc so that all elements are initialized to 0
39     for(int i = 0; i < N; i++){
40         flag[s[i]]++;
41     }
42     for(int i = c/2; i>=0&& c-i<=v; i--){
43         //start from the middle part of the flag array,
44         //making it easier to avoid out-of-bounds.
45         if(flag[i]&&flag[c-i]){
46             //within bound
47             if(i == c-i&&flag[i]<2) continue;
48             //the same number
49             *a = i, *b = c-i;
50             free(flag);
51             //printf("%d + %d = %d\n",i, c-i, c);
52             return 1;
53         }
54     }
55     free(flag);
56     //printf("F\n");
57     return 0;
58 }
59
60 static int seed = 0;
61
62 void random(int *s, int N, int V){
63     if(!seed){
64         srand((unsigned)time(NULL));
65         //use the current time as the random seed
66         seed = 1;
67         //avoid using the same seed and the same array!
68     }
69     for(int i = 0; i < N; i++) s[i] = rand()%V + 1;
70     //[1,V]
71 }
72
73 int main(){
74     int a, b, tick, K, v = 100000;
75     //fixed v in this code
76     double duration, total;
77     int test[]={1000, 5000, 10000, 20000, 40000, 60000, 80000, 100000};
78     //giving an array of N
79     for(int i = 0; i < 8; i++){
80         //testing different Ns
81         N = test[i];
82         /*****
83         Algorithm 1
84         *****/
85         K = 120000/N+1;
86         //setting a proper K to keep total time in a reasonable range
87         c = 1;
88         //providing boundary cases to demonstrate the time complexity!

```



```

89     start = clock();
90     //repeating the algorithm for K times
91     for(int j = 0; j < K; j++) {random(S, N, V); find1(S, N, V, c, &a, &b);}
92     stop = clock();
93     total = (double)(stop - start) / CLK_TCK, tick = stop - start, duration =
(double)total/K;
94     //output the time and N and K
95     printf("Algorithm1, N = %6d, K = %5d, Tick: %6d, Total_time: %6.3lf,
Duration: %.6lf\n", N, K, tick, total, duration);
96
97     /*****
98     Algorithm 2
99     *****/
100    K = 1000000/N+1;
101    //this one is faster, so repeating more times to improve accuracy
102    c = N/2;
103    //also providing boundary cases to demonstrate the time complexity
104    start = clock();
105    for(int j = 0; j < K; j++) {random(S, N, V); find2(S, N, V, c, &a, &b);}
106    stop = clock();
107    total = (double)(stop - start) / CLK_TCK, tick = stop - start, duration =
(double)total/K;
108    printf("Algorithm2, N = %6d, K = %5d, Tick: %6d, Total_time: %6.3lf,
Duration: %.6lf\n\n", N, K, tick, total, duration);
109    }
110    system("pause");
111    return 0;
112 }

```

Declaration:

I hereby declare that all the work done in this project titled **Performance Measurement** ($A + B$) is of my independent effort.

1. The experiment fixed $V = 100000$. Testing different V values would reveal how Algorithm 2's space complexity ($O(V)$) and time complexity $O(N + V)$ impacts performance. [↗](#)